Architecture proxy n8n — Gestion déportée de Grist

Architecture proxy n8n — Gestion déportée de Grist

Document technique — contexte, choix d'architecture, état actuel

---

Contexte

Les widgets EMILE sont des interfaces React hébergées sur **GitHub Pages** (`stiiig.github.io`). En mode normal, ils tournent dans un **iframe embarqué dans Grist** : c'est le plugin Grist qui leur fournit l'accès aux données via `grist.docApi` (une API JavaScript injectée dans l'iframe).

Ce mode fonctionne bien pour les utilisateurs qui ouvrent Grist. Il ne couvre pas deux besoins clés :

- **Accès orienteur standalone** — permettre à un orienteur d'accéder à ses candidats via un lien personnel, sans ouvrir Grist (token OCC)
- **Formulaires publics** — `ajout-etablissement`, `creation-compte-orienteur` accessibles sans compte Grist

L'objectif : faire fonctionner ces widgets **hors iframe**, en appelant directement l'API Grist depuis le navigateur.

Pourquoi on ne peut pas appeler Grist directement depuis le browser

Deux obstacles rendent l'appel direct impossible :

1. CORS

Le navigateur applique la politique *Same-Origin* : un script chargé depuis `stiiig.github.io` ne peut pas faire de requête vers `grist.incubateur.dnum.din.developpement-durable.gouv.fr` sauf si ce dernier répond avec le header `Access-Control-Allow-Origin: *`. L'instance Grist interne ne le fait pas — les appels sont bloqués avant même d'atteindre le serveur.

2. Clé API exposée

L'API Grist REST exige un `Authorization: Bearer <clé>` sur chaque requête. Si on appelle Grist depuis le browser, la clé est visible dans le JS du bundle ou dans les DevTools — n'importe qui peut la récupérer et lire ou modifier toute la base de données.

> **Note** : ce problème sera partiellement résolu par une **clé de service** (compte applicatif Grist avec permissions restreintes à lecture/écriture sur les tables EMILE uniquement, sans accès admin). En attente de provisionnement côté infra. Même avec une clé de service, le problème CORS reste entier — le proxy reste nécessaire.

Solution : proxy n8n

n8n est la plateforme d'automatisation déjà déployée sur l'infra. Elle est accessible depuis internet et gère le CORS côté serveur.

```
Browser (GitHub Pages)
      │
      │  GET https://n8n.incubateur.dnum.../webhook/grist-proxy?
table=CANDIDATS&token=OCC
      │  (pas de clé API, pas de contrainte CORS)
      ▼
n8n (accessible publiquement)
      │  vérifie HMAC-SHA256 du token
      │  Authorization: Bearer <clé API>   ← jamais vue par le browser
      │  GET https://grist.incubateur.dnum.../api/docs/{docId}/tables/CANDIDATS/records
      ▼
Grist (réseau interne, inaccessible depuis internet)
      │  {"records": [...]}
      ▼
n8n → ajoute Access-Control-Allow-Origin: *
      ▼
Browser → données disponibles dans React
```

Le proxy joue quatre rôles : 1. **Contournement CORS** — il appelle Grist server-side, sans contrainte cross-origin 2. **Isolation de la clé API** — elle reste dans les credentials n8n, jamais dans le bundle JS 3. **Passerelle réseau** — Grist est sur le réseau interne, n8n peut l'atteindre, le browser non 4. **Vérification des tokens** — HMAC-SHA256 vérifié côté n8n, secret jamais exposé

Architecture actuelle — workflows n8n actifs

Workflows `grist-proxy-get` + `grist-proxy-post` — proxy principal Grist

Deux workflows n8n distincts sur le **même path** `/webhook/grist-proxy`, routés par méthode HTTP. Ensemble, ils gèrent toutes les interactions avec l'API Grist : lectures, écritures, pièces jointes, FAQ. Correspondent tous les deux à `NEXT_PUBLIC_GRIST_PROXY_URL`.

> ℹ️ n8n permet d'avoir deux workflows sur le même path webhook avec des méthodes différentes (GET et POST). Le browser appelle toujours la même URL — n8n route vers le bon workflow automatiquement.

`grist-proxy-get` — lecture + téléchargement (GET)

```
Webhook GET (?table=X&filter=JSON  ou  ?table=X  ou  ?attachId=Y)
    │
    ▼
IF query.attachId est présent
    ├─ True  → GET /attachments/{id}/download → Respond Binary
    └─ False → GET /tables/{table}/records[?filter=...]  → Respond JSON
```

Couvre toutes les requêtes de lecture : tables de données (`CANDIDATS` , `ACCOMPAGNANTS` …), tables de référence (`DPTS_REGIONS` , `ETABLISSEMENTS` …), métadonnées Grist (`_grist_Tables` , `_grist_Tables_column` , `_grist_Attachments`).

`grist-proxy-post` — écritures Grist + upload de pièces jointes (POST)

```
Webhook POST
    │
    ▼
IF Content-Type contient "text/plain"
    │
    ├─ True  → Code (JSON.parse du body texte → extrait table, _action, id, fields)
    │           │
    │           IF _action = "update"
    │           ├─ True  → HTTP PATCH /tables/{table}/records → Respond JSON {}
    │           └─ False → HTTP POST  /tables/{table}/records → Respond JSON {
retValues: [newId] }
    │
    └─ False → (multipart/form-data — upload de pièce jointe)
                HTTP POST /attachments (Form-Data, champ upload, Bearer Auth)
                    → Respond JSON { data: "[42]" } + CORS header
```

Clé : éviter le preflight CORS OPTIONS

- `text/plain;charset=UTF-8` sans header custom → *simple CORS request* (spec WHATWG Fetch)
- `multipart/form-data` sans header custom → également une *simple CORS request*

À l'inverse, `Content-Type: application/json` ou la méthode `PATCH` depuis le navigateur déclenchent un preflight OPTIONS que n8n ne gère pas. La solution : n'utiliser que `text/plain`

côté `rest.ts` pour toutes les écritures. Le champ `_action` dans le body JSON permet à n8n de distinguer AddRecord d'UpdateRecord.

Workflow `occ-generate` — génération de token OCC orienteur

```
Webhook GET /webhook/occ-generate?rowId=X
    │
    ▼
Code (extrait rowId depuis $json.query.rowId)
    │
    ▼
Crypto HMAC-SHA256 (credential EMILE HMAC Secret)
    │
    ▼
Code (construit token = rowId.HMAC et URL → liste-candidats?token=X.HMAC)
    │
    ▼
Respond to Webhook { rowId, token, url }  ← frontend reçoit l'URL immédiatement
    │
    ▼
HTTP PATCH ACCOMPAGNANTS.Lien_validation = url  ← sauvegarde en background dans Grist
```

OCC = **Orienteur Compte Créer**. Génère un token d'accès orienteur. Appelé automatiquement par : - `creation-compte-orienteur` après AddRecord dans `ACCOMPAGNANTS` - `inscription-candidat` après l'inscription d'un·e candidat·e (pour générer le lien "Voir mes candidats" de l'orienteur)

Clé : Respond to Webhook AVANT le PATCH Grist — n8n continue l'exécution après avoir répondu. Le frontend reçoit le token sans attendre la mise à jour Grist.

Workflow `occ-validate` — vérification et activation du compte orienteur

```
Webhook GET /webhook/occ-validate?token=X.HMAC
    │
    ▼
Code (extrait rowId + sig du token)
    │
    ▼
Crypto HMAC-SHA256 (credential EMILE HMAC Secret)
    │
    ▼
```

```
IF sig === HMAC calculé
    ├─ False → Respond { "status": "invalid" }
    └─ True  → HTTP GET ACCOMPAGNANTS?filter={"id":[rowId]}
                │
                ▼
               Code (extrait Compte_valide)
                │
                ▼
               IF Compte_valide === "Oui"
                ├─ True  → Respond { "status": "already_validated" }
                └─ False → HTTP PATCH Compte_valide = "Oui"
                              → Respond { "status": "ok", "nom": "..." }
```

Appelé par `validation-compte` au chargement. Vérifie le token, lit le compte dans `ACCOMPAGNANTS`, et passe `Compte_valide` de `"En attente"` à `"Oui"` si le token est valide et que le compte n'était pas encore activé.

Workflow `occ-list` — liste des candidats d'un orienteur

```
Webhook GET /webhook/occ-list?token=X.HMAC
    │
    ▼
Code (extrait rowId + sig du token)
    │
    ▼
Crypto HMAC-SHA256 (credential EMILE HMAC Secret)
    │
    ▼
IF sig === HMAC calculé
    ├─ False → Respond { "status": "invalid" }
    └─ True  → HTTP GET ACCOMPAGNANTS?filter={"id":[rowId]}
                │  (récupère le nom de l'orienteur)
                ▼
               HTTP GET CANDIDATS?filter={"Responsable_candidat":[rowId]}
                │  (récupère les candidats rattachés)
                ▼
               Code (formate la réponse)
                │  Pour chaque candidat : id, prenom, nom, email, tel,
                │  genre, dateNaissance, reference, createdAt, nationalite, statut,
                │  eligibilite, aie, territoireDepart, niveauLangueElig,
                │  regulariteSituation, precariteLogement, volontariteMobilite
```

```
                │  (⚠️ ne jamais exposer $Age — âge calculé côté frontend depuis
dateNaissance)
                ▼
               Respond {
                 "status": "ok",
                 "orienteurNom": "Prénom Nom",
                 "candidats": [...]
               }
```

Appelé par `liste-candidats` et `fiche-candidat` (pour le switcher de candidats et l'auto-sélection). Le token identifie l'orienteur de manière cryptographiquement vérifiée, sans session ni cookie.

> **Champ** `Responsable_candidat` : colonne Reference dans CANDIDATS pointant vers ACCOMPAGNANTS. Grist renvoie un rowId entier — le filtre doit utiliser l'entier, pas le nom affiché.

Workflow `occ-get-candidat` — fiche d'un candidat pour un orienteur

```
Webhook GET /webhook/occ-get-candidat?token=X.HMAC&id=ROW_ID
    │
    ▼
Code (extrait rowId orienteur + sig du token)
    │
    ▼
Crypto HMAC-SHA256 (credential EMILE HMAC Secret)
    │
    ▼
IF sig === HMAC calculé
    ├─ False → Respond { "status": "invalid" }
    └─ True  → HTTP GET CANDIDATS?filter={"id":[id]}
                │  (vérifie que le candidat appartient bien à cet orienteur)
                ▼
               Respond {
                 "status": "ok",
                 "candidat": { id, prenom, nom, ... (tous les champs) }
               }
```

Appelé par `fiche-candidat` en mode orienteur (`?token=OCC&id=ROW_ID` ou via session localStorage + auto-sélection). Vérifie le token OCC **et** que le candidat demandé appartient bien à cet orienteur.

Workflow `occ-request-link` — renvoi du lien de connexion orienteur

```
Webhook POST /webhook/occ-request-link  { email }
    │
    ▼
Code (normalise email en minuscules)
    │
    ▼
HTTP GET ACCOMPAGNANTS?filter={"Email":[email]}
    │
    ▼
Code (extrait rowId du premier enregistrement trouvé)
    │
    ▼
IF rowId trouvé
    ├─ False → Respond { "status": "not_found" }
    └─ True  → HTTP GET /webhook/occ-generate?rowId=X
                │
                ▼
               Code (extrait le token depuis la réponse)
                │
                ▼
               Code (construit l'URL liste-candidats?token=X.HMAC)
                │
                ▼
               Respond { "status": "ok", "url": "https://.../liste-candidats/?
token=..." }
```

Appelé par `recuperer-lien-connexion`. Renvoie un lien pointant vers `liste-candidats` (espace de travail de l'orienteur·rice).

> ⚠️ **Attention au champ `url` de occ-generate** : ce workflow retourne une `url` qui pointe vers `validation-compte` (lien créé à la genèse du compte). Il ne faut **pas** utiliser ce champ directement — le Code node reconstruit l'URL manuellement avec le token extrait, en ciblant `liste-candidats`.

Workflow `occ-request-validation-link` — renvoi du lien de validation orienteur

```
Webhook POST /webhook/occ-request-validation-link  { email }
    │
    ▼
(idem OCC-REQUEST-LINK jusqu'à la résolution du rowId)
```

```
    │
    ▼
IF rowId trouvé
    ├─ False → Respond { "status": "not_found" }
    └─ True  → HTTP GET /webhook/occ-generate?rowId=X
                │
                ▼
               Respond { "status": "ok", "url": $json.url }
               ← utilise directement l'url validation-compte retournée par occ-generate
```

Appelé par `recuperer-lien-validation`. Contrairement à OCC-REQUEST-LINK, utilise directement le champ `url` de occ-generate car il pointe déjà vers `validation-compte`.

Session orienteur — localStorage

`liste-candidats` et `fiche-candidat` partagent une session via la clé `emile_occ_token` dans le localStorage du navigateur :

Événement	Action
Token OCC validé par `occ-list` ou `occ-get-candidat`	`localStorage.setItem("emile_occ_token", token)`
Token invalide ou expiré	`localStorage.removeItem("emile_occ_token")`
Déconnexion (bouton logout)	`localStorage.removeItem("emile_occ_token")` + reload
`fiche-candidat/` sans `?id=` mais token en session	Mode orienteur, auto-sélection du candidat avec le `id` le plus élevé (le plus récent)
`fiche-candidat/` ou `liste-candidats/` sans token	Affiche “Connexion requise” avec bouton vers `recuperer-lien-connexion`
Token présent mais invalide/expiré	Affiche “Lien invalide”

Ce que ça permet concrètement

Sans modifier Grist, sans plugin, sans compte Grist côté utilisateur :

Fonctionnalité	Workflow	Page appelante
Lire les tables de référence (départements, établissements…)	GET `grist-proxy?table=X`	Tous les widgets
Charger les métadonnées colonnes (types, options dropdowns)	GET `grist-proxy?table=_grist_Tables` + `_grist_Tables_column`	`fiche-candidat`
Afficher les pièces jointes (noms)	GET `grist-proxy?table=_grist_Attachments`	`fiche-candidat`
Télécharger une pièce jointe	GET `grist-proxy?attachId=42` → binaire	`fiche-candidat`
Uploader une pièce jointe	POST `grist-proxy` multipart/form-data	`fiche-candidat`
Inscrire un·e candidat·e (AddRecord)	POST `grist-proxy` text/plain `{ _action:"add" }`	`inscription-candidat`
Générer un token OCC orienteur	GET `occ-generate?rowId=X`	`inscription-candidat`, `creation-compte-orienteur`
Activer un compte orienteur	GET `occ-validate?token=X.HMAC`	`validation-compte`
Lister les candidats d'un orienteur	GET `occ-list?token=X.HMAC`	`liste-candidats`, `fiche-candidat`
Afficher la fiche d'un candidat (orienteur)	GET `occ-get-candidat?token=X.HMAC&id=ROW_ID`	`fiche-candidat`
Renvoyer le lien de connexion orienteur	POST `occ-request-link { email }`	`recuperer-lien-connexion`
Renvoyer le lien de validation de compte	POST `occ-request-validation-link { email }`	`recuperer-lien-validation`
Charger la FAQ depuis Grist	GET `grist-proxy?table=FAQ`	Tous les widgets (FAQPanel)

Sécurité des tokens

Format (commun à tous les workflows OCC)

```
token = rowId + "." + HMAC-SHA256(rowId.toString(), SECRET)
```

- **rowId** — identifiant de l'enregistrement Grist (entier) : rowId dans `ACCOMPAGNANTS` pour les tokens OCC orienteur, rowId dans `ACCOMPAGNANTS` pour OCC-VALIDATE
- **HMAC-SHA256** — signature cryptographique avec un secret partagé entre les workflows n8n
- **SECRET** — stocké uniquement dans les credentials n8n (credential `EMILE HMAC Secret` de type Crypto), jamais dans le code ni dans le repo

> ⚠️ Les workflows `occ-generate`, `occ-validate`, `occ-list` et `occ-get-candidat` utilisent **tous le même credential** `EMILE HMAC Secret`. Si une isolation renforcée est souhaitée (token orienteur ≠ token validation), utiliser deux credentials secrets distincts.

Propriétés

Propriété	Valeur
Forgeable sans le secret	❌ Non
Expiration	❌ Permanent (pas d'expiration)
Révocable	⚠️ Uniquement en changeant le secret (invalide **tous** les tokens)
Lié à un enregistrement spécifique	✅ Oui (rowId dans le token)
Partage de session entre pages	✅ Oui (localStorage `emile_occ_token`)

Workflows archivés (ne plus utiliser)

Workflow	Raison
`grist-generate`	Générait un magic link par candidat — remplacé par le token OCC orienteur

Le mode "magic link candidat" (`fiche-candidat?token=rowId.HMAC` côté candidat) a été retiré. L'accès à la fiche se fait désormais exclusivement via token OCC orienteur.

> **Historique** : l'ancien workflow `grist - GET fiche-candidat` (GET sur path `/webhook/grist`) gérait les lectures Grist. Il a été renommé et déplacé vers `grist-proxy-get` (GET sur path `/webhook/grist-proxy`) lors de la refonte du proxy. Il est toujours actif sous ce nouveau nom.

Limitations et chantiers ouverts

Clé de service Grist

Actuellement la clé API configurée dans n8n est une clé personnelle. Une **clé de service** (compte applicatif, permissions minimales, rotation sans impact humain) est nécessaire pour la production. En attente de provisionnement côté infra Grist.

Expiration des tokens

Les tokens OCC sont permanents. Pour des données sensibles, une expiration (date butoir dans le token, vérifiée par n8n) pourrait être ajoutée ultérieurement.

Fichiers concernés dans le repo

Fichier	Rôle
`src/lib/grist/rest.ts`	Client REST — implémente `GristDocAPI` via proxy n8n (`grist-proxy`)
`src/lib/grist/init.ts`	Détecte `NEXT_PUBLIC_GRIST_PROXY_URL` et bascule en mode REST
`src/lib/grist/meta.ts`	Charge métadonnées colonnes via `_grist_Tables`
`src/components/AttachmentField.tsx`	Gestion pièces jointes (affichage, download fetch+blob, upload)
`src/components/FAQPanel.tsx`	Panneau FAQ — charge les questions depuis Grist via `grist-proxy`
`src/app/widgets/emile/inscription-candidat/page.tsx`	Formulaire d'inscription — AddRecord via `text/plain` + génération token OCC orienteur
`src/app/widgets/emile/creation-compte-orienteur/page.tsx`	Formulaire création compte orienteur — AddRecord ACCOMPAGNANTS + génération token OCC
`src/app/widgets/emile/validation-compte/page.tsx`	Activation compte orienteur — `occ-validate?token=X.HMAC`
`src/app/widgets/emile/liste-candidats/page.tsx`	Espace orienteur — liste ses candidats via `occ-list` , session localStorage, logout

Fichier	Rôle
`src/app/widgets/emile/fiche-candidat/page.tsx`	Fiche candidat — token OCC + `occ-get-candidat`, session localStorage, auto-sélection candidat le plus récent
`src/app/widgets/emile/recuperer-lien-connexion/page.tsx`	Renvoi du lien liste-candidats par email — `occ-request-link`
`src/app/widgets/emile/recuperer-lien-validation/page.tsx`	Renvoi du lien validation-compte par email — `occ-request-validation-link`
`src/app/emile/dev/links/page.tsx`	Page de test — liste tous les widgets avec leurs URL et paramètres
`docs/rest-mode.md`	Config n8n pas-à-pas avec tous les pièges rencontrés